FLIZOW · TECH BRIEF FOR SENIOR WEB DEV

Client Access Health

A live integration-access dashboard for the client profile — surfaced as colored dots beside Quick Links. This document captures the design intent, the build paths we considered, and the open decisions that need senior input before we commit.

Audience. Senior web developer scoping the backend + integration work.
Read time. ~10 minutes.
Status. Pre-build. Decision points flagged at the end of each section.

What we're solving

An agency loses access to a client's Google Ads, Meta Business, GA4, Klaviyo, etc. all the time — the client revokes a manager invite, an OAuth token rots, the client's CTO leaves and their account gets cleaned out. The agency usually finds out three days later when a campaign needs editing and they can't get in.

We want a single place on each client's profile where the team can see, at a glance, whether the agency has live access to every platform we manage for that client.

> **THE VISIBLE FEATURE**
>
> Inside the Quick Links card on the client profile's About tab, add an "Access health" sub-section. Each row = one platform, with a colored status dot.

The visual — three dot states

Pinned to a small set so the row reads at a glance:

- **Live** — the agency currently has access. Green.
- **Lost** — access has been revoked or the token no longer authorizes. Red.
- **Pending request** — the agency has asked the client for access; waiting on them to grant it. White (outlined).

No "expiring soon," no "stale check," no fourth color — keep it binary plus a request-in-flight state. Anything more nuanced belongs in row metadata (last checked, who has it on the agency side), not in the dot itself.

Placement

Inside the existing **Quick Links** card on the About tab. Quick Links is currently a flat list of URLs (link to GA dashboard, Klaviyo admin, etc.). The Access Health rows live as a sub-section inside the same card — same logical grouping, since each platform has both a URL *and* an access state.

Layout intention: Quick Links section header, then the existing link list, then a divider, then "Access health" with one row per integration. The row's status dot is the leading visual; the platform name is the row label; secondary metadata (last checked, who's authorized) hangs to the right.

Three build paths

We discussed three implementation strategies, ranging from "ship the UI now, prove the value, then automate" to "build full end-to-end automation per platform up front." The honest split:

PATH A · FULL AUTOMATIC

OAuth + scheduled API checks per platform

For each platform (Google Ads first, then Meta, GA4, Klaviyo, etc.):

1. OAuth flow — the agency signs in once from Flizow, the platform issues a refresh token.
2. Token storage — secure backend (Cloud Functions + secret manager, or equivalent). Cannot live in Firestore in plaintext.
3. Scheduled job — every N hours, walk every (client × platform) pair, ask the platform: *"do we still have access to this account?"* Flip dot to red on failure.
4. Per-client mapping — each client record needs the platform account ID (Google Ads MCC, Meta Business ID, etc.).
5. Error handling — token refresh, scope changes, rate limits, Google revoking apps under suspicious traffic, etc.

Estimated cost: ~2–3 weeks per platform for a senior dev, plus ongoing maintenance.

Worth it if: losing access is a recurring, expensive pain in the team's daily work AND the agency wants this as a competitive differentiator.

PATH B · UI NOW, AUTOMATION LATER (RECOMMENDED)

Manual marking as the v1, swap to API in v2

Build the data model + UI now, with the AM marking the dot manually. Show "last checked" timestamp + a freshness indicator when the check is older than ~30 days. When automation is eventually added, the data shape doesn't change — the dot's setter just flips from "human" to "scheduled API call."

Estimated cost: ~1–2 days for the UI, data model, and edit flow.

Worth it because: proves the feature earns its weight in the daily flow before we invest multi-week infrastructure. The visual signal is in the app immediately. No backend needed yet.

PATH C · HALF-AUTOMATIC

"Check now" button per row

Each row has a button that opens the platform in a new tab at a known URL. The AM verifies access there, comes back, and the row's last-checked time auto-updates. Status still set by the AM, but verification is real and the friction is minimal.

Estimated cost: ~1 day on top of Path B's UI.

Worth it because: feels semi-real without backend work. Good middle ground if Path B feels too theatrical and Path A is too expensive.

Two realities about "live"

1. "Live" is never instant

Even with Path A, there's a polling delay. If the client revokes access at 9:00 AM and the system polls every 6 hours, the dot doesn't go red until ~3:00 PM. The product copy should reflect this — "Last verified 2 hours ago" is more honest than "Live."

Polling cadence is a tunable trade-off: 1-hour polling × 5 platforms × 50 clients = 12,000 API calls/day per platform. Most platforms have rate limits that handle this fine, but it does eat quota. Webhook-based notifications (where supported) are the better long- term pattern.

2. "Pending request" is always manual

No platform exposes a "we asked the client to add us" state. That signal lives only in the agency's head ("we emailed Sarah on Tuesday asking her to add us as a manager"). Even Path A has at least one human-set state.

Implication: the data model needs to support an explicit `status: 'pending'` with a manual setter. The auto-detect layer only writes `'live'` or `'lost'`; it never overwrites

`'pending'` until the platform itself reports active access (which would naturally clear the pending state to live).

Suggested data model

Roughly:

Field	Type	Notes
`id`	`string`	Stable id for the row.
`clientId`	`string`	FK to `Client.id`.
`platform`	`string`	Platform key — e.g. `'google-ads'`, `'meta'`, `'ga4'`, `'klaviyo'`.
`platformLabel`	`string`	Display name. May be a workspace-level lookup so editing the label is one-touch.
`accountId`	`string?`	Platform-specific account id (Google Ads MCC, Meta Business ID, etc.). Required for Path A; optional for Paths B/C.
`status`	`'live' | 'lost' | 'pending'`	The dot.
`lastCheckedAt`	`ISO timestamp`	When the status was last set / verified. Drives "checked X ago" display.
`setBy`	`'system' | memberId`	Who flipped it. Lets the UI show a small "auto" badge vs. the AM's avatar.
`notes`	`string?`	Optional. e.g. *"Lost when their CTO left in March."*

Suggested store location: top-level `data.clientIntegrations` array on `FlizowData`, parallel to `contacts` / `quickLinks` / `notes`. Same patterns: flat rows, clientId lookup at render time.

Decision points for senior dev review

D1 — Which path?

Reviewer call: Path A, Path B, or Path C. Recommendation in this brief is Path B with Path A as a follow-up scoped to one platform (Google Ads first).

D2 — If Path A, where does the OAuth + scheduler live?

Flizow today is frontend-only (Vite/React + Firestore via the client SDK). Token storage and scheduled jobs need a backend. Options:

- **Firebase Cloud Functions** — already in the Firebase project; lowest infrastructure cost; cron jobs via scheduled functions; secret manager via Firebase config.
- **Standalone Node service** (Cloudflare Workers, Fly.io, Render, etc.) — more flexibility, more ops work.
- **Third-party integration platform** (Nango, Pipedream, Cobalt) — fastest to ship; recurring per-connection cost; less control over the connection flow's UX.

D3 — Which platforms first, and in what priority?

Each platform is its own multi-week build. Cap the v1 scope at 1–2 platforms and iterate. Suggested ordering by frequency-of-pain:

1. **Google Ads** — manager (MCC) invites get revoked often when the client has multiple agencies churning through.
2. **Meta Business Manager** — same churn pattern plus role-based weirdness.
3. **GA4** — third because lost GA4 access is rarer but harder to recover.
4. **Klaviyo / Mailchimp / etc.** — niche tools per client; treat as v3.

D4 — Polling cadence + webhook strategy

Hourly polling × 50 clients × 5 platforms = 240,000 API calls/month. Most platforms tolerate this; some have stricter quotas. Where webhook notifications exist (Google does for some Ads events, Meta has webhooks for Business changes), prefer those over polling to drop quota use to near-zero. Pure polling is fine as a v1.

D5 — Per-client account ID entry UX

Path A requires each client to have its platform account IDs in Flizow. We need a UX for entering and editing those IDs (likely a small "Add platform" modal in the same Quick Links card area). Light data-entry burden once per client per platform, but real.

What we'd like from senior

1. **Sanity-check on path.** Is Path B → A staged the right call? Or does the team have appetite to commit to Path A for Google Ads up front?
2. **Backend decision.** If we're going down the Path A route eventually: are we comfortable with Firebase Cloud Functions, or does the architecture warrant a separate Node service?
3. **OAuth integration model.** One agency-wide OAuth grant per platform (the agency authorizes Flizow once; Flizow re-uses that grant across all clients), or per-client OAuth grants (every client connects independently)? The first scales better; the second is more secure / easier to revoke selectively.
4. **Time estimate.** Realistic engineering days for Path A on Google Ads alone, end-to-end (OAuth + storage + scheduler + UI hookup + error handling)?
5. **Maintenance posture.** Once shipped, who owns keeping the integrations alive when platforms change their APIs? The agency, an external dev, ongoing retainer with you?

Document version 1 — pre-build brief. Update after senior review with the path call and timeline.